



Trading Signals

Rollout Plan

Microservice + Dashboard Integration Proposal

Table of Contents

Right-click the TOC and select "Update Field" to refresh page numbers

Table of Contents	1
Executive Summary.....	2
Proposal Overview	3
Current Capabilities	3
Product Requirements	3
Proposed Architecture	3
Service Architecture	4
API Contract (v1).....	4
Signal Payload Schema	5
Signal Logic & Core Pipeline.....	5
Free vs. Pro Access Logic	6
Free Tier.....	6
Pro Tier	6
Response Extension.....	6
Frontend Integration Plan.....	7
Execution Phases	7
New Tasks	7
Definition of Done	8

Executive Summary

This proposal outlines the delivery of a standalone Trading Signals microservice that integrates our existing Pulse-TradingAgents AI engine with the Pulse dashboard. The service will emit normalized trading signals (ticker, signal type, confidence, reasoning) and serve them through a new dashboard interface with feed, filters, and free-vs-pro access logic.

The initiative leverages our existing AI infrastructure, including the 5-tier rating system (Buy, Overweight, Hold, Underweight, Sell) and structured agent outputs so as to deliver a production-ready signals product within 7-12 days across three execution phases.

Timeline	Service Type	Signal Types
7-12 days	Standalone Microservice	Buy / Overweight / Hold / Underweight / Sell

Proposal Overview

Our TradingAgents system already produces high-quality trading decisions through a multi-agent pipeline. This proposal bridges the gap between raw AI output and user-facing product by creating a dedicated microservice that normalizes, persists, and serves signals at scale.

Current Capabilities

- TradingAgentsGraph.propagate() returns 5-tier ratings via process_signal
- Structured outputs for Trader and Portfolio Manager agents enable consistent signal parsing
- Pulse frontend supports new tab/menu routes with existing patterns
- pulse-feed-service provides reference architecture for CRUD, filtering, and caching

Product Requirements

- Standalone feed with real-time ticker and signal updates
- Free-tier limits with cooldown lock and subscription tier preparation
- Filterable signal history with watchlist tracking

Proposed Architecture

Service Architecture

The proposed architecture separates signal generation, normalization, storage, and serving into a dedicated microservice that integrates with existing Pulse infrastructure:

Component	Description
SignalScheduler	Orchestrates signal generation on configurable cadence
TradingAgentsRunner	Invokes TradingAgentsGraph.propagate() for each ticker
SignalNormalizer	Transforms raw AI output into canonical signal records
SignalsDB	Persistent storage for normalized signals and audit trail
RedisCache	Caching layer for hot signal feeds and latest data
TradingSignalsAPI	REST endpoints serving filtered signals to frontend
pulse-auth-service	Entitlement verification and tier enforcement

The SignalScheduler orchestrates execution, the TradingAgentsRunner invokes the AI pipeline, and the SignalNormalizer transforms raw outputs into canonical signal records. The TradingSignalsAPI serves filtered data to the frontend while enforcing entitlement rules through the pulse-auth-service.

API Contract (v1)

The pulse-trading-signals-service exposes the following REST endpoints:

Endpoint	Description
GET /signals-ms/signals	Filtered signal feed with pagination (ticker, type, date range)
GET /signals-ms/signals/latest	Most recent signal per ticker for real-time panel
GET /signals-ms/tickers	Tracked tickers list with activity statistics
GET /signals-ms/stream	SSE live stream for real-time updates (Phase 2)
GET /signals-ms/health	Service health check endpoint

Signal Payload Schema

Field	Type	Description
id	UUID	Unique signal identifier
ticker	String	Asset ticker symbol
asset_type	Enum	stocks crypto
signal_type	Enum	buy overweight hold underweight sell
confidence	Float	Signal confidence score (0-1)
time_horizon	String	Expected holding period
price_target	Decimal	Target price level
entry_price	Decimal	Recommended entry price
stop_loss	Decimal	Risk stop-loss level
position_sizing	String	Recommended position size
reasoning_summary	Text	AI-generated thesis summary
generated_at	Timestamp	Signal creation time
source_run_id	String	Audit trace to raw execution

Signal Logic & Core Pipeline

The signal generation pipeline processes configurable ticker watchlists (stocks and crypto) on a scheduled cadence:

- **Input:**Configurable ticker watchlist and execution cadence
- **Execution:**Run `TradingAgentsGraph.propagate(ticker, trade_date, asset_type)`
- **Mapping:**PM rating maps to canonical `signal_type`; `TraderProposal` + PM fields populate `entry_price`, `stop_loss`, `position_sizing`, `time_horizon`, and `price_target`
- **Confidence:**Heuristic v1 combines PM decisiveness, debate convergence, and optional token statistics
- **Deduplication:**No duplicate signals for same ticker + `signal_type` + unchanged thesis within configured window
- **Audit:**Normalized record and raw source preserved for debugging

Free vs. Pro Access Logic

Entitlement gating is enforced server-side using auth claims from the pulse-auth-service:

Free Tier

- Limited feed windows (maximum N signal views per hour/day)
- Limited live duration (10 minutes per day)
- On exhaustion: return `lock_state` with `cooldown_ends_at` timestamp

Pro Tier

- Unlimited signal history access
- Real-time SSE stream
- Personal watchlist tracking

Response Extension

All API responses include an entitlement object: { tier, remaining_views, reset_at, locked, cooldown_ends_at }

Frontend Integration Plan

The pulse-frontend integration follows existing patterns for tab navigation and API hooks:

- New sidebar tab: trading-signals
- TradingSignalsFeed component with feed list, ticker filter, signal-type filter, and watch toggle
- Lock/cooldown UI for free users with clear upgrade prompts
- Optional live mode via SSE (Phase 2)
- API hook service following existing hooks pattern

Execution Phases

Phase	Duration	Deliverables
Phase 0	1-2 days	Specs, API contract freeze, Kanban board setup
Phase 1	3-5 days	Microservice MVP: run TA, store normalized signals, REST read API
Phase 2	2-4 days	Frontend tab, feed, filters, entitlement lock UI
Phase 3	2-3 days	Live SSE stream, monitoring, production hardening

Total estimated timeline: 7-12 days from spec freeze to production deployment.

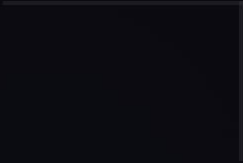
New Tasks

- Epic: Trading Signals v1 (separate microservice)
- Service scaffold + CI/CD + dockerization
- TradingAgents adapter + normalized signal schema
- Signals DB schema + migrations + retention policy
- REST endpoints (/signals, /signals/latest, /tickers)

- Scheduler + watchlist execution logic
- Free/Pro entitlement enforcement + cooldown contract
- Dashboard tab + feed + filters
- Live updates (SSE) phase 2
- QA/observability/runbook + production rollout checklist

Definition of Done

- New service deployed and queryable with stable API contract
- Signals generated from TradingAgents and persisted with full audit trail
- Dashboard has new Trading Signals menu with filters and lock/cooldown UX
- Free/Pro limits enforced server-side and reflected in all API responses
- Kanban board has clean Trading Signals epic with sequenced tasks, owners, and status



Pulse

Trade with the Market's Pulse

pulsenow.io

© 2026 Pulse. All rights reserved.

